

Encrypted Keylogger with Simulated Exfiltration

Tools: Python, pynput, cryptography, base64, Flask

◆ Introduction

This project demonstrates the working of a **Proof-of-Concept (PoC) keylogger** that encrypts all captured keystrokes and simulates secure data exfiltration to a localhost server. It is intended for **educational and ethical cybersecurity research purposes only**. Keyloggers are commonly used in cyber-attacks, and understanding how they work helps in developing defensive mechanisms.

◆ Abstract

The project involves capturing user keystrokes using Python and the pynput library, encrypting the data using symmetric AES encryption via cryptography.fernet, and then simulating secure transmission to a locally hosted server using Flask. Logs are timestamped and stored in encrypted form to mimic stealth and secure data handling. It includes basic persistence (auto-start on boot for Windows) and a kill switch to stop logging immediately when a specific key is pressed (ESC key).

◆ Tools Used

Tool	Purpose
Python 3.x	Programming language
pynput	Capturing keyboard inputs
cryptography	Fernet encryption (symmetric AES)
base64	Encoding encrypted data for transmission
Flask	Simulating a remote server (localhost)
requests	Sending data from logger to the server

◆ Steps Involved in Building the Project

1. Keylogging

- Used `pynput.keyboard.Listener` to continuously capture keystrokes.
- Keystrokes are processed and stored as strings, even for special keys (e.g., ENTER, SPACE).

2. Encryption

- Generated a symmetric key using Fernet.
- Encrypted keystroke logs using this key and stored them in a file (`log.txt`).

3. Timestamping

- Each captured keystroke is logged with a precise date and time stamp for traceability.

4. Simulated Exfiltration

- Used `base64` to encode encrypted logs.
- Sent data using `requests.post()` to a simulated Flask server running on `localhost`.

5. Server Setup (Flask)

- A local Flask server listens for incoming POST requests and saves the received logs to a separate file (`received_logs.txt`).

6. Kill Switch

- Implemented a key-based stop trigger (ESC key).
- On trigger, stops the keylogger and sends any remaining logs to the server.

7. Optional Persistence (Windows)

- Provided a simple `.bat` file to auto-copy the keylogger into the Windows startup folder.

◆ Conclusion

This project provides a compact, encrypted keylogger with simulated data exfiltration capabilities, helping users understand the inner workings of such cybersecurity threats. It demonstrates how simple scripts can capture, encrypt, and transfer sensitive information silently, reinforcing the importance of endpoint security, behavioral monitoring, and encryption awareness.